

Advanced Logic Design

RISC-V CPU on FPGA

Gioele Sesso

Project goal

RISC-V CPU

The aim of the project was to recreate a RISC-V processing unit on an FPGA programmed via Vivado in VHDL language.

The fundamental challenge was set by the (single) pipeline structure.

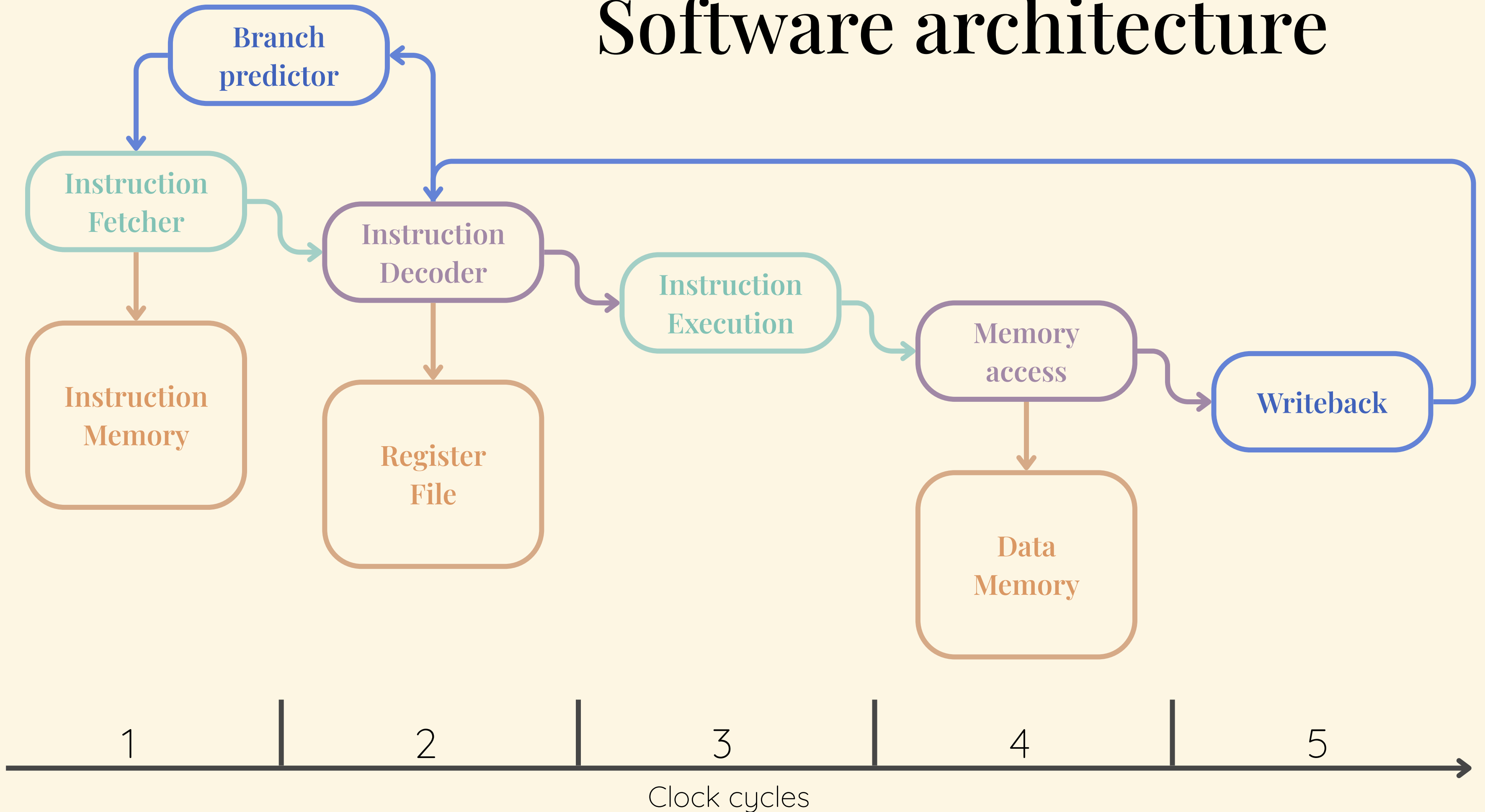
FPGA approximation

- BRAM Memory access in 1 cycle
- Registers file instantaneous access
- Reduced clock frequency (100MHz)

Highlights of the architecture

- Variant RV32I, integer operations with 32 registers
- Pipeline with 5 stages
- Single bit BTNF (Backward Taken, Forward Not) branch predictor
- Early branch detection

Software architecture



Instruction execution flow

Logic to execution steps

1. Load the instruction word at the address of **PC**
2. Decode the instruction
 - a. Detect the type of instruction from the **opcode** and evaluate the **ALU_OP** signal
 - b. Fetch the **signed IMM** value from the instruction and the register value of RS1
 - c. Trigger the **write-back flag** to store the result
3. Execute the operation with the given operands
4. Send **through the pipeline** the store register operation
5. Store the integer in the right address of the **register file**

X3 = 3

ADDI X3, X0, 3

0x 00300193

000000000011 - 00000 - 000 - 00011 - 0010011

IMM[11:0] - RS1 - FUNC3 - RD - OP CODE

Instruction execution flow

Pipeline implications

The general concept of it is that while a first operation is in the execution step, a second one is getting decoded and a third one is being copied from the instruction memory.

Control and data hazards

Control hazards are caused by instructions that are interrupting the regular flow (PC increment), these can be jumps or branches (conditional jumps).

Data hazards happen when a needed **resource** like a register value or a memory address is being used by the preceding or following instruction.

Jumps

JAL operations are managed in Instruction Fetch stage to **not introduce** any **bubble**. JALR operations are executed in the Instruction Decode stage (as it needs a register access) to introduce a **single bubble**.

Branches

Branch operations are executed with a single-bit **BTBN branch predictor** that **changes the flow** if the movement is backwards otherwise will ignore it until the condition is executed. Since they are executed in the Instruction Decode stage they introduce **one bubble**.

(Next slide)

Instruction execution flow



Data hazards

Data hazards are mainly scenarios in which a write and a read are too close for a complete 5-stages execution.

These are commonly defined as:

- Write-After-Read, i.e. LOAD to X1 $\Rightarrow$ ADD to X1
- Read-After-Write, i.e. ADD to X2 $\Rightarrow$ SUB from X2
- Write-After-Write, i.e. ADD to X3 $\Rightarrow$ SUB to X3

Data bypassing – not implemented

Data bypassing can be implemented in a pipeline to prevent them.

For example, being able to send directly an output from Execute to Decode or to the same Execute stage, can mitigate the problem allowing for instantaneous access to the needed resources.

In the last implementation version this was omitted to keep the structure solid and straightforward.

Implementation – OLED case

Peripherals communication

An OLED screen with a basic graphic has been used to show the working CPU.

The display is connected via **SPI protocol** to the unit as a **memory-mapped** device.

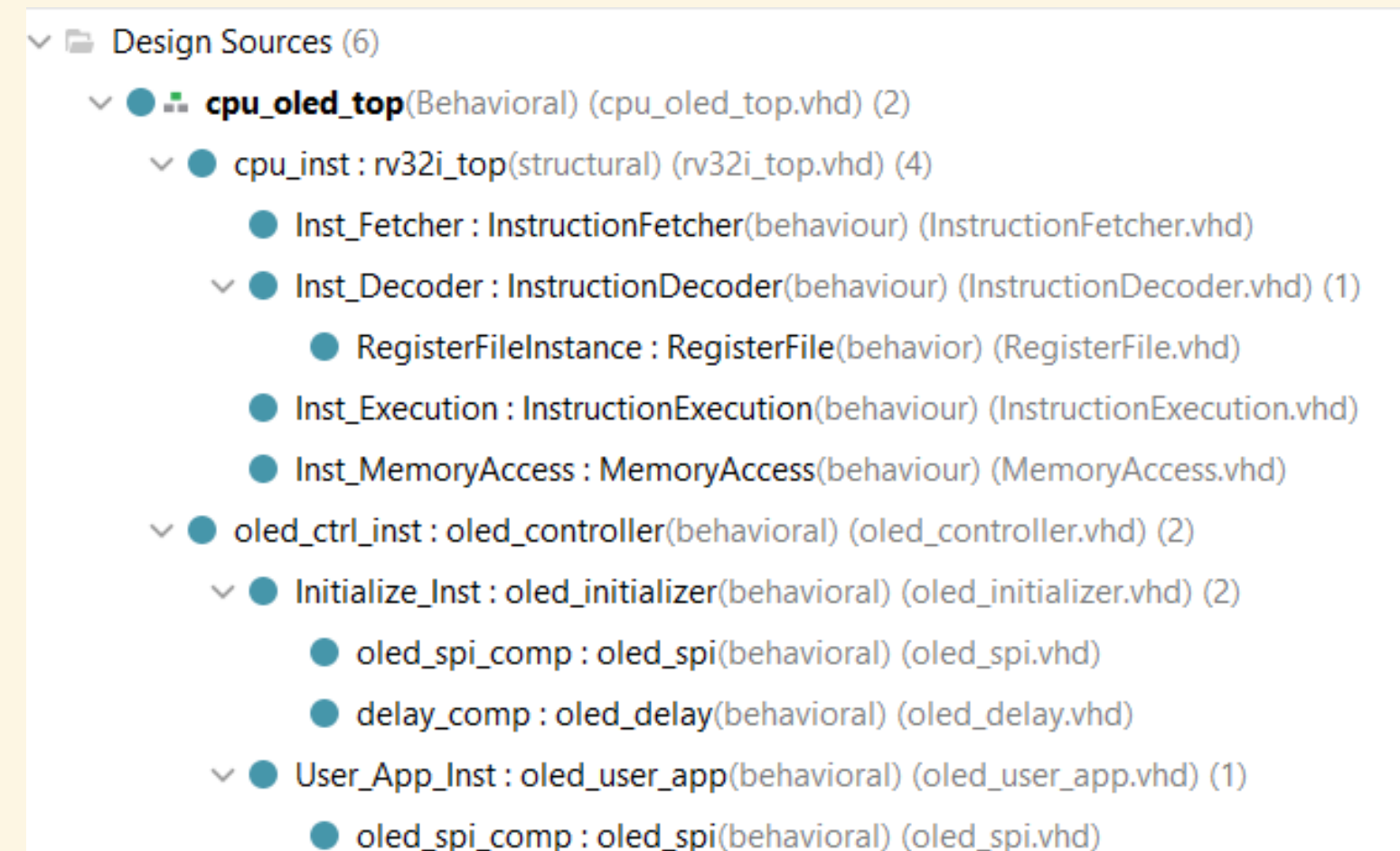
An **abstraction layer** is introduced with a screen **controller** that manage the real wires part while accepting higher-level commands.

The graphic is updated with a **specific register** reachable at the address **000F0004**; 32 bits are used to specify **x** (7 downto 0) and **y** (15 downto 8) **coordinates** of the square.

Code structure

The **top-level entity** contains the RV32I implementation and the OLED controller.

in the controller there are the components to **initialize** the device, start the SPI communication and then the User App that contains the **FSM** to manage the animation.

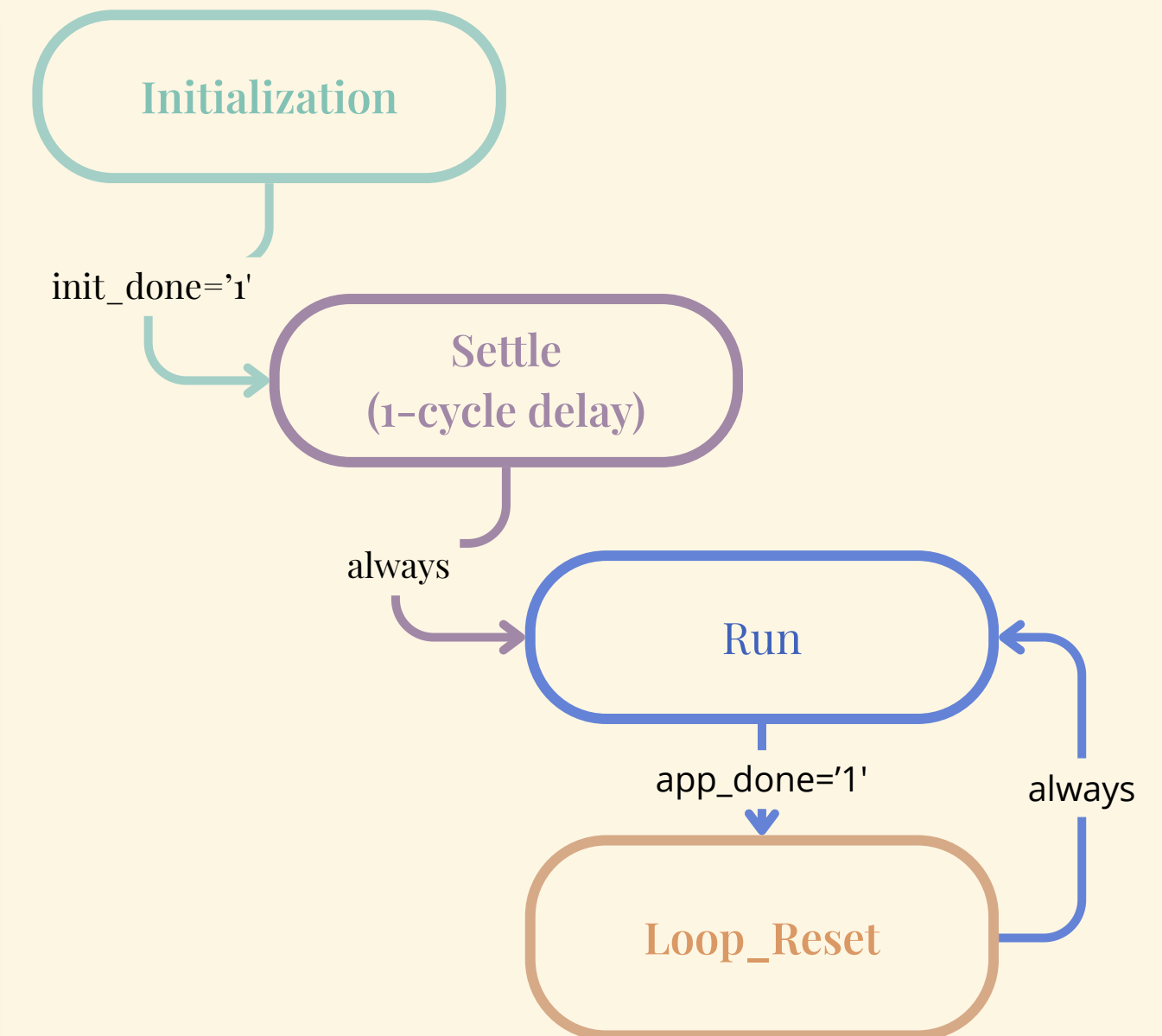


Implementation – OLED case

RV32I with MMIO code

```
1 BASE_ADDR = 0x000F0000 // memory-mapped OLED command register
2 X, Y      = 2, 2        // starting position of the box
3 STEP_FWD  = +1
4 STEP_BACK = -1
5 DELAY_COUNT = 56000    // cycles to wait after each pixel move
6
7 function draw(x, y):
8     cmd = (y << 8) | x
9     write cmd to BASE_ADDR + 4 // MMIO write → OLED redraws box at (x,y)
10
11 function wait(cycles):
12     repeat cycles times:
13         cycles = cycles - 1 // busy-wait / software delay
14
15 function move(steps, dx, dy):
16     repeat steps times:
17         X = X + dx
18         Y = Y + dy
19         draw(X, Y)
20         wait(DELAY_COUNT)
21
22 // ===== Main loop: trace a rectangle, forever =====
23 loop forever:
24     move(120, STEP_FWD, 0) // LEG 1: RIGHT (X: 2 → 122)
25     move(24, 0, STEP_FWD)  // LEG 2: DOWN (Y: 2 → 26)
26     move(120, STEP_BACK, 0) // LEG 3: LEFT (X: 122 → 2)
27     move(24, 0, STEP_BACK)  // LEG 4: UP (Y: 26 → 2)
```

OLED Controller FSM



One refresh per loop

Implementation – Performance

CPU Performance – post implementation analysis

Design Timing Summary

Setup

Worst Negative Slack (WNS): 0.217 ns
Total Negative Slack (TNS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 22613

Hold

Worst Hold Slack (WHS): 0.068 ns
Total Hold Slack (THS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 22613

Pulse Width

Worst Pulse Width Slack (WPWS): 3.750 ns
Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0
Total Number of Endpoints: 3775

All user specified timing constraints are met.

Utilization	Name	Clocks (W)	Signals (W)	Data (W)	Clock Enable (W)	Set/Reset (W)	Logic (W)	I/O (W)
▼ 0.018 W (14% of total)	cpu_oled_top							
> 0.014 W (12% of total)	cpu_inst (rv32i_top)	0.009	0.003	0.003	<0.001	<0.001	0.002	<0.001
0.002 W (2% of total)	Leaf Cells (9)							
> 0.002 W (1% of total)	oled_ctrl_inst (oled_controller)	<0.001	<0.001	<0.001	<0.001	<0.001	0.001	<0.001

Name	Slice LUTs (53200)	Slice Registers (106400)	F7 Muxes (26600)	F8 Muxes (13300)	Slice (13300)	LUT as Logic (53200)	LUT as Memory (17400)	Bonded IOB (200)	BUFGCTRL (32)
▼ cpu_oled_top	4364	1783	1256	575	1419	2316	2048	8	2
> cpu_inst (rv32i_top)	4223	1624	1256	575	1354	2175	2048	0	1
> oled_ctrl_inst (oled_controller)	141	159	0	0	65	141	0	0	0

The diagram illustrates a 4x4 grid of cells, labeled X0Y0 to X3Y3. The grid is divided into four quadrants by a central horizontal and vertical line. Each quadrant contains a 2x2 sub-grid of cells. The cells are color-coded: X0Y0 is blue, X0Y1 is green, X0Y2 is red, and X0Y3 is yellow. The cells are labeled with their coordinates in the top-left corner of each cell. The grid is surrounded by a thick black border. The diagram is a technical drawing of a grid structure, likely for a game or simulation.